Cómo funciona el modelo nuevo (Random Forest), paso a paso

Explicación del script `Modelo_Random_Forest.R`.

Qué hace en una frase: lee la base diaria de demanda eléctrica de GRAN BS.AS. con el clima de 4 estaciones, entrena un *Random Forest* que estima la demanda media del día (en MW) a partir de la temperatura, la humedad y el mes, y mide qué tan bien funciona en días que no usó para aprender.

Base que usa: `TP_Metodologia/2_Base_de_datos/Base_de_Datos.xlsx`, hoja `Base madre` (2.007 días, del 01/01/2021 al 30/06/2026, 11 columnas). Se arma con `Construir_Base_de_Datos.R`, en la misma carpeta.

> Los números de este documento salen de correr el script tal cual, con R 4.x y `ranger`, sobre esa base. Con la misma semilla (42), cualquiera que lo corra obtiene los mismos números.

Paso 0. Cargar las librerías (líneas 5 a 13)

Líneas 6 a 13 de `Modelo_Random_Forest.R`:

```
 6 | paquetes <- c("readxl", "ranger", "ggplot2", "DiagrammeR")
 7 | paquetes_faltantes <- paquetes[!paquetes %in% installed.packages()[, "Package"]]
 8 | if (length(paquetes_faltantes) > 0) install.packages(paquetes_faltantes)
 9 |
10 | library(readxl)
11 | library(ranger)
12 | library(ggplot2)
13 | library(DiagrammeR)
```

El script revisa qué paquetes faltan, instala sólo esos y después los carga con `library()`. Para qué sirve cada uno:

Paquete	Para qué se usa
`readxl`	Leer el Excel de la base.
`ranger`	Entrenar el Random Forest (es una implementación rápida).
`ggplot2`	Hacer los gráficos.
`DiagrammeR`	Dibujar uno de los árboles del bosque.

Paso 1. Leer y preparar los datos (líneas 15 a 36)

Líneas 16 a 19 de `Modelo_Random_Forest.R`:

```
 16 | archivo <- file.choose() #Seleccionamos el archivo Excel con nuestra base de
datos
 17 | df <- read_excel(archivo, sheet = "Base madre") #Leemos nuestro dataset
 18 |
 19 | df_clean <- df[!is.na(df$demanda_electrica_neta_media_MW), ] #Eliminamos las
filas con valores faltantes en la demanda
```

1. Línea 16: `file.choose()` abre una ventana para que el usuario elija el Excel a mano: `TP_Metodologia/2_Base_de_datos/Base_de_Datos.xlsx`.
2. Línea 17: se lee la hoja `Base madre`.
3. Línea 19: se descartan los días sin dato de demanda. En esta base no hay ninguno, así que quedan los 2.007 días.

Las variables de entrada (*features*) (líneas 21 a 32)

El modelo usa 8 variables para estimar la demanda:

Variable	Qué es
`temperatura_media_C_san_fernando`	Temperatura media del día en San Fernando (°C)
`humedad_relativa_media_pct_san_fernando`	Humedad relativa media en San Fernando (%)
`temperatura_media_C_ezeiza`	Temperatura media en Ezeiza (°C)
`humedad_relativa_media_pct_ezeiza`	Humedad en Ezeiza (%)
`temperatura_media_C_buenos_aires_obs`	Temperatura media en Buenos Aires Observatorio (°C)
`temperatura_media_C_la_plata`	Temperatura media en La Plata (°C)
`humedad_relativa_media_pct_la_plata`	Humedad en La Plata (%)
`mes_del_anio`	Mes, de 1 (enero) a 12 (diciembre)

La variable a estimar (*variable objetivo*, línea 32) es `demanda_electrica_neta_media_MW`: la potencia media del día en la región GRAN BS.AS., en MW.

La base trae 10 columnas además de la fecha, pero el modelo usa 8. Quedan afuera la fecha y la humedad de Buenos Aires Observatorio (`humedad_relativa_media_pct_buenos_aires_obs`). Las otras tres estaciones entran con temperatura y humedad.

Líneas 35 a 36 de `Modelo_Random_Forest.R`:

```
 35 | df_modelo <- df_clean[, c(variable_objetivo, features)]
 36 | df_modelo <- na.omit(df_modelo)
```

Se arma una tabla sólo con la demanda y las 8 variables, y `na.omit` borra las filas con algún dato vacío. En esta base no hay ninguno, así que siguen siendo 2.007 filas.

Paso 2. Separar entrenamiento y prueba (líneas 38 a 42)

Líneas 39 a 42 de `Modelo_Random_Forest.R`:

```
 39 | set.seed(42) #Fijamos una semilla para poder reproducir la division
 40 | indices_train <- sample(seq_len(nrow(df_modelo)), size = floor(0.8 *
nrow(df_modelo)))
 41 | train <- df_modelo[indices_train, ]
 42 | test <- df_modelo[-indices_train, ]
```

- `set.seed(42)` fija la semilla del azar, así el sorteo da siempre lo mismo.
- Se sortean al azar el 80 % de los días para **entrenar** (1.605 días) y el 20 % restante queda para **probar** (402 días).
- El modelo aprende sólo con `train`. Con `test` se mide qué tan bien estima días que no vio.

Como el sorteo es al azar, los días de prueba quedan mezclados en todo el período 2021-2026, y sus vecinos (el día anterior y el siguiente) pueden estar en entrenamiento.

Paso 3. Entrenar el Random Forest (líneas 45 a 55)

Líneas 46 a 55 de `Modelo_Random_Forest.R`:

```
 46 | formula_rf <- reformulate(features, response = variable_objetivo)
 47 |
 48 | rf_model <- ranger(
 49 |   formula = formula_rf,
 50 |   data = train,
 51 |   num.trees = 100,
 52 |   max.depth = 10,
 53 |   seed = 42,
 54 |   importance = "impurity"
 55 | ) #Le pasamos los parámetros y entrenamos el modelo
```

`reformulate` (línea 46) arma la fórmula `demanda ~ temp_SF + hum_SF + ... + mes`, que le dice a R qué estimar y con qué.

Qué es un árbol de decisión

Un árbol de regresión estima la demanda haciendo preguntas de sí o no sobre las variables, por ejemplo: "¿la temperatura en Buenos Aires es menor o igual a 13 °C?". Según la respuesta baja por una rama u otra, y vuelve a preguntar. Al llegar al final de una rama (una *hoja*), la estimación es el **promedio de la demanda** de los días de entrenamiento que cayeron en esa hoja.

Para elegir cada pregunta, el árbol prueba cortes posibles y se queda con el que separa los días en dos grupos con la demanda más parecida dentro de cada grupo (el que más reduce la varianza).

Qué es un Random Forest

Un solo árbol es inestable: con datos apenas distintos puede cambiar mucho. Un Random Forest entrena **muchos árboles distintos** y **promedia** sus estimaciones. Cada árbol es distinto por dos fuentes de azar:

1. **Cada árbol ve una muestra distinta de días.** Se sortean días de entrenamiento con reposición (algunos se repiten y otros quedan afuera). Se llama *bootstrap*.
2. **En cada pregunta, el árbol sólo puede elegir entre unas pocas variables sorteadas.** `ranger` usa por defecto la raíz cuadrada de la cantidad de variables: con 8 variables, sortea 2 en cada corte (`mtry = 2`). Así los árboles no repiten siempre las mismas preguntas.

Los parámetros del script

Parámetro	Valor	Qué significa
`num.trees`	100	Cantidad de árboles del bosque.
`max.depth`	10	Cada árbol puede hacer como máximo 10 preguntas seguidas desde la raíz hasta una hoja.
`seed`	42	Semilla para que el bosque salga igual cada vez.
`importance`	`"impurity"`	Pide calcular cuánto aporta cada variable (ver Paso 7).
`mtry` (por defecto)	2	Variables sorteadas en cada corte.
`min.node.size` (por defecto)	5	Un nodo con menos de 5 días no se sigue partiendo.

Qué devuelve el entrenamiento

Al imprimir `rf_model`, `ranger` informa un error *OOB* (*out of bag*): cada árbol se prueba con los días que su sorteo dejó afuera. Da un **R^2 OOB de 0,78**: el modelo explica cerca del 78 % de la variación de la demanda dentro del entrenamiento.

Paso 4. Estimar los días de prueba (líneas 57 a 59)

Líneas 58 a 59 de `Modelo_Random_Forest.R`:

```
58 | y_real_MW <- test[[variable_objetivo]]
59 | y_pred_rf <- predict(rf_model, data = test)$predictions
```

Para cada uno de los 402 días de prueba, cada uno de los 100 árboles da una estimación y el bosque devuelve el promedio. Quedan dos vectores: la demanda real (`y_real_MW`) y la estimada (`y_pred_rf`).

Paso 5. Calcular las métricas (líneas 61 a 79)

Líneas 62 a 66 de `Modelo_Random_Forest.R`:

```
62 | errores <- y_real_MW - y_pred_rf
63 | errores_absolutos <- abs(errores)
64 |
65 | mae_rf <- mean(errores_absolutos)
66 | rmse_rf <- sqrt(mean(errores^2))
```

El error de cada día es **real menos estimado**. Positivo quiere decir que el modelo estimó de menos.

Métrica	Cómo se calcula	Qué dice	Resultado
MAE global	Promedio de los errores sin signo	"En promedio nos equivocamos por tantos MW."	**382 MW**
RMSE global	Raíz del promedio de los errores al cuadrado	Como el MAE, pero castiga más los errores grandes.	**471 MW**
MAE en picos	MAE sólo en los días de demanda más alta	Qué tan bien anda en los días críticos.	**406 MW**
Subestimación en picos	Promedio real de los picos menos promedio estimado	Si da positivo, el modelo se queda corto en los picos.	**344 MW**
Desvío del error absoluto (línea 78)	`sd()` de los errores sin signo	Cuánto varía el tamaño del error de un día a otro.	**277 MW**
Coef. de variación del error (línea 79)	Desvío / MAE × 100	El desvío en relación al MAE.	**72 %**

Como referencia, la demanda media de la base ronda los 6.000 MW, así que un MAE de 382 MW equivale a un error medio de alrededor del 6 %.

Cómo se definen los "picos" (líneas 68 a 75)

Líneas 69 a 75 de `Modelo_Random_Forest.R`:

```
69 | limite_alto <- as.numeric(quantile(y_real_MW, 0.90))
70 | indices_picos <- which(y_real_MW >= limite_alto)
71 | picos_reales <- y_real_MW[indices_picos]
72 | picos_predichos_rf <- y_pred_rf[indices_picos]
73 |
74 | mae_picos_rf <- mean(abs(picos_reales - picos_predichos_rf))
75 | subestimacion_rf <- mean(picos_reales) - mean(picos_predichos_rf)
```

Se toma el percentil 90 de la demanda **real de los días de prueba**: 7.318 MW. Los días de prueba con demanda igual o mayor son "picos": 41 días. En esos días la demanda real promedio fue 7.793 MW y el modelo estimó en promedio 7.449 MW, o sea 344 MW de menos.

Paso 6. Imprimir la tabla (líneas 81 a 118)

El script imprime las métricas en la consola con `cat` y `sprintf`. Al final agrega la cantidad de picos y los promedios real y estimado en picos.

Hay un detalle de formato: la fila "MAE en Picos (Top 10% MW)" sale con un `%` al final, porque el script agrega `%` a toda métrica cuyo nombre contenga ese símbolo (línea 107). Ese valor está en MW, no en porcentaje.

Paso 7. Gráficos (líneas 120 a 274)

Gráfico 1: real contra estimado (líneas 123 a 143)

Toma los primeros 100 días de prueba y dibuja dos líneas: la demanda real y la estimada. Los días de prueba están en orden de fecha, pero no son consecutivos (son los que salieron en el sorteo), así que el eje horizontal es un número de muestra y no una fecha.

Gráfico 2: dispersión y picos (líneas 145 a 167)

Cada punto es un día de prueba: la demanda real en el eje horizontal y la estimada en el vertical. La línea roja punteada es la diagonal: un punto sobre ella sería una estimación perfecta. Los picos (top 10 %) van en naranja. Si los puntos naranjas quedan por debajo de la diagonal, el modelo se queda corto en los días de demanda alta, que es lo que muestra la métrica de subestimación.

Gráfico 3: demanda contra el clima y el mes (líneas 169 a 210)

Tres paneles, con la demanda real (azul) y la estimada (violeta):

- contra la **humedad de La Plata**;
- contra la **temperatura de Buenos Aires Observatorio**. Acá se ve la forma de U: la demanda sube con el frío (calefacción) y con el calor (aire acondicionado), y es mínima cerca de 18-20 °C;
- contra el **mes**, donde se ven los picos de invierno y de verano.

Gráfico 4: importancia de las variables (líneas 212 a 229)

Con `importance = "impurity"`, `ranger` suma, para cada variable, cuánto redujo la varianza cada vez que un árbol la usó para cortar. Cuanto más alta la barra, más usó el bosque esa variable. El orden que sale es:

1. Temperatura Buenos Aires Observatorio
2. Temperatura San Fernando
3. Temperatura La Plata
4. Temperatura Ezeiza
5. Mes del año
6. Humedad San Fernando
7. Humedad Ezeiza
8. Humedad La Plata

Las cuatro temperaturas son, por lejos, lo más importante. Como las cuatro estaciones están cerca y sus temperaturas se mueven juntas, el bosque reparte la importancia entre ellas: no quiere decir que una estación "cause" más demanda que otra.

Gráfico 5: uno de los árboles (líneas 231 a 274)

Línea 232 de `Modelo_Random_Forest.R`:

```
232 | arbol_ejemplo <- ranger::treeInfo(rf_model, tree = 1) #Seleccionamos el primer
árbol del modelo
```

`treeInfo` devuelve la tabla de nodos del primer árbol: qué variable pregunta cada nodo, en qué valor corta, a qué nodos lleva cada respuesta y, en las hojas, la estimación. Ese árbol tiene 463 nodos, demasiados para leerlos, así que la función `agregar_nodo` (líneas 242 a 268) recorre el árbol desde la raíz y dibuja sólo los **3 primeros niveles**:

- cada nodo intermedio muestra la pregunta, por ejemplo `temperatura_media_C_buenos_aires_obs <= 13.05`;
- cada flecha dice "Sí" o "No";
- donde el árbol sigue más abajo se dibuja "...".

El texto se arma en lenguaje DOT (el formato de Graphviz) (líneas 236 a 271) y `DiagrammeR::grViz` lo dibuja (líneas 273 y 274).

En el primer árbol del bosque, la primera pregunta es si la temperatura de Buenos Aires es ≤ 13,05 °C (frío o no frío). Después vuelve a preguntar por la temperatura (≤ 8,05 °C del lado frío, ≤ 25,25 °C del lado templado/cálido). Así el árbol va armando por tramos la misma U que se ve en los datos.

Resumen del recorrido

1. Cargar paquetes.
2. Leer la base y quedarse con la demanda y 8 variables de clima y mes.
3. Sortear 80 % de los días para entrenar y 20 % para probar.
4. Entrenar 100 árboles, cada uno con una muestra distinta de días y de variables.
5. Estimar los días de prueba promediando los 100 árboles.
6. Medir el error: MAE 382 MW, RMSE 471 MW, y 344 MW de subestimación en los picos.
7. Graficar real contra estimado, la relación con el clima, la importancia de las variables y un árbol de ejemplo.